

Team 40 Project Report: RAG and Context Engineering

CSE 234 Project 1

Yunxuan Li

Abstract

This project builds a single-turn retrieval-augmented generation (RAG) system over the RapidFire AI OSS documentation corpus. The final submission is a deterministic line-aware BM25 pipeline that retrieves source spans, assembles context under the 2,000-token budget, and generates grounded answers with a TritonAI/OpenAI-compatible chat model. I also created 22 hand-labeled golden question-answer pairs and used both local retrieval sweeps and RapidFire AI OSS multi-config experiments to compare chunking, embedding, retrieval, reranking, prompting, and generation choices.

1 Data Creation Methodology

I created 22 hand-labeled golden Q&A pairs in `golden_qa_pairs.json`, exceeding the required minimum of 20. Each example follows the released validation schema: `question_id`, `question`, `reference_answer`, and `source_evidence`. The source evidence is not a loose page-level citation; it records concrete file-names and line spans so that retrieval metrics can score source overlap directly.

The questions were designed to cover the kinds of user intents that a documentation assistant should handle: factual lookup, conceptual explanation, procedural setup, comparative design questions, and troubleshooting. I scanned the RST documentation for high-value concepts such as config group generation, RAG specifications, vector store modes, search configuration, prompt managers, dashboards, IC Ops, and installation failure modes. For each candidate, I wrote an answer only after verifying it against the source file and narrowing the supporting evidence to the smallest useful line span.

Quality control had three parts. First, I checked that each question required the corpus rather than general prior knowledge. Second, I avoided over-broad evidence spans that would make retrieval artificially easy. Third, I calibrated the answer style against

the released validation examples without copying validation answers or hardcoding validation-specific logic. I did not add extra custom LLM judge metrics beyond the provided retrieval evaluator and released LLM-as-judge script; instead, I focused on precise evidence annotations and later used the official metrics to guide iteration.

2 Final Pipeline Architecture

The final automated pipeline is implemented in `main.py` and `src/`. It reads a JSON question set, loads the documentation corpus, retrieves relevant line-aware chunks, serializes them into a bounded context, generates an answer, and writes the required output fields: `question_id`, `answer`, `retrieved_context`, and `sources`. Figure 1 summarizes the control flow.

The corpus loader automatically locates `sourcedocs/` and reads `.rst`, `.txt`, and `.md` files. The selected chunking strategy uses overlapping line windows with `chunk_lines = 54` and `chunk_overlap = 24`. Line-aware chunks preserve exact source spans, which is important because the retrieval evaluator rewards overlap between predicted and gold evidence lines.

The final retriever is a custom BM25 lexical retriever, not a dense embedding retriever. It tokenizes API identifiers, camel case, numeric terms, and conservative stems, then adds boosts for exact API names, code identifiers, phrases, headings, and filename matches. This choice was empirical: the documentation corpus contains many precise API and configuration names, and lexical matching produced better line-span retrieval than the dense RapidFire configurations on the released validation set.

The final retrieval settings are `top_k = 5`, `candidate_pool = 48`, and a **1700-token retrieved-context budget**. This leaves room for the user question and prompt while keeping the full prompt under the project limit of 2,000 context tokens per query.

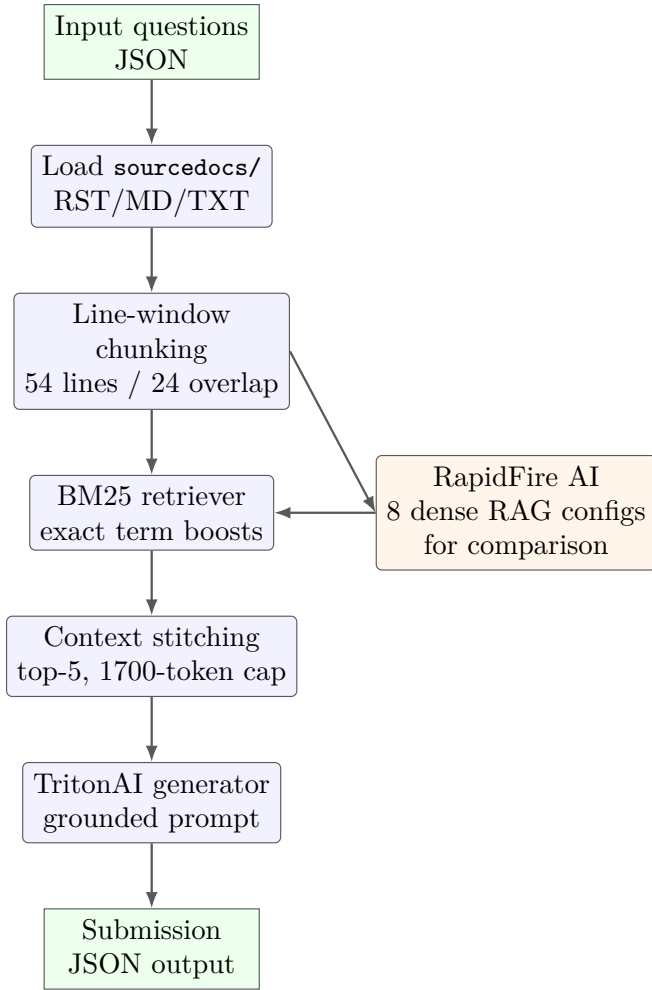


Figure 1: Final RAG pipeline and the parallel RapidFire experiment track.

The context serializer adds labels such as [Source: ragspecs.rst lines 103-156], which helps both the generator and the faithfulness judge.

Generation is implemented in `src/llm_client.py`. It uses an OpenAI-compatible chat completions endpoint, so it works with TritonAI through `OPENAI_API_KEY`, `GENERATOR_BASE_URL`, and `GENERATOR_MODEL`. The prompt asks the model to answer only from retrieved context, be concise but complete, and avoid unsupported claims. A deterministic extractive fallback exists for debugging if the API is unavailable, but the submitted validation output was generated with the TritonAI-compatible model and contains no fallback warning text.

3 Experimentation Methodology

I used two complementary experiment tracks. The first was a local BM25 sweep over the exact deterministic retriever used by the final pipeline. This tested `chunk_lines`, `chunk_overlap`, `top_k`, and `candidate_pool` with the released span-overlap retrieval evaluator. The second was a RapidFire AI OSS experiment using `RFLangChainRagSpec` and `run_evals()` to compare eight dense RAG configurations side by side.

The RapidFire workflow is implemented in `scripts/run_rapidfire_experiments.py`. It loads `configs/rapidfire_experiment_configs.json`, constructs a RapidFire dataset, defines preprocess and postprocess functions, computes source-overlap metrics, wraps the generator with `RFOpenAIAPIModelConfig`, and launches the configs through RapidFire’s multi-config API. The RAG spec uses `DirectoryLoader`, `RecursiveCharacterTextSplitter`, HuggingFace CPU embeddings, FAISS vector storage, similarity or MMR retrieval, optional cross-encoder reranking, and either a concise or structured grounded-answer prompt.

These eight configurations were chosen to satisfy the project requirement for at least eight meaningfully different config combinations while keeping Triton API cost controlled. They vary the major RAG knobs requested by the assignment: chunking, embedding model, retrieval strategy, reranking, prompting, and generator settings. The design is not a full factorial grid; instead, it covers representative low-cost contrasts, such as MiniLM versus BGE, similarity versus MMR, no reranking versus cross-encoder reranking, and concise versus structured prompting.

4 Results and Analysis

Table 2 shows the local BM25 retrieval sweep. The best final pipeline config was `bm25_lines54_overlap24_top5`, with **Retrieval Score = 0.7739**. It was selected because it balanced precision and recall better than the smaller chunks, which had high recall but more irrelevant evidence, and better than the largest chunks, which improved precision but lost evidence coverage.

The RapidFire dense experiments told a different but useful story. Figure 2 shows that Run 5, `BGE-small + 256/32 chunks + similarity`

Table 1: Eight RapidFire RAG configurations. Blank values in the logs are intentional: `fetch_k` and `lambda` apply only to MMR, and `top_n` applies only when reranking is enabled. All runs use FAISS, `api-llama-4-scout`, temperature 0.0, and max tokens 300.

Run	Embedding	Chunk	Search	$k/\text{fetch}/\lambda$	Reranker	top_n	Prompt
1	MiniLM-L6-v2	256/32	similarity	5 / - / -	none	-	concise
2	MiniLM-L6-v2	256/32	MMR	5 / 24 / 0.35	none	-	structured
3	MiniLM-L6-v2	512/64	similarity	7 / - / -	cross-encoder	5	concise
4	MiniLM-L6-v2	512/64	MMR	7 / 32 / 0.35	cross-encoder	5	structured
5	BGE-small	256/32	similarity	5 / - / -	none	-	structured
6	BGE-small	384/64	MMR	5 / 24 / 0.40	none	-	concise
7	BGE-small	512/64	similarity	7 / - / -	cross-encoder	5	structured
8	BGE-small	512/96	MMR	7 / 32 / 0.35	cross-encoder	5	concise

Table 2: Local BM25 retrieval sweep on the released validation set.

Config	F1@5	P@5	R@5	Score
54/24, k5	0.7292	0.6315	0.9611	0.7739
96/24, k5	0.7324	0.6667	0.8944	0.7645
80/20, k5	0.6953	0.6074	0.8944	0.7324
66/18, k5	0.6746	0.5796	0.9167	0.7236
54/14, k4	0.6568	0.5444	0.9278	0.7097
54/14, k5	0.6531	0.5407	0.9278	0.7072
42/10, k5	0.6449	0.5304	0.9389	0.7047
36/12, k5	0.6330	0.5037	0.9667	0.7011

search + no reranker + structured prompt, had the best source F1@5 at **0.5363**; Run 7 had the best reference-token recall at **0.4774**. BGE generally improved recall over MiniLM, but MMR and cross-encoder reranking did not consistently improve source-span metrics. This was the main surprise: a simpler BGE similarity configuration beat several more complex reranked configurations. My interpretation is that cross-encoder reranking helped some answer-token coverage but sometimes moved line spans away from the exact source evidence needed by the evaluator.

The final validation output from the selected deterministic pipeline achieved **Precision@5 = 0.6315**, **Recall@5 = 0.9611**, **F1@5 = 0.7292**, and **Retrieval Score = 0.7739**. On the released LLM judge metrics, it achieved **Correctness = 0.9556**, **Faithfulness = 0.9778**, **Completeness = 4.6667/5**, and **Partial Generation Score = 0.9556**. These results explain the final architecture choice: dense retrieval was valuable for design exploration, but the deterministic BM25 pipeline was stronger for the exact source-span scoring target and more robust for hidden-test grading without requiring model downloads.

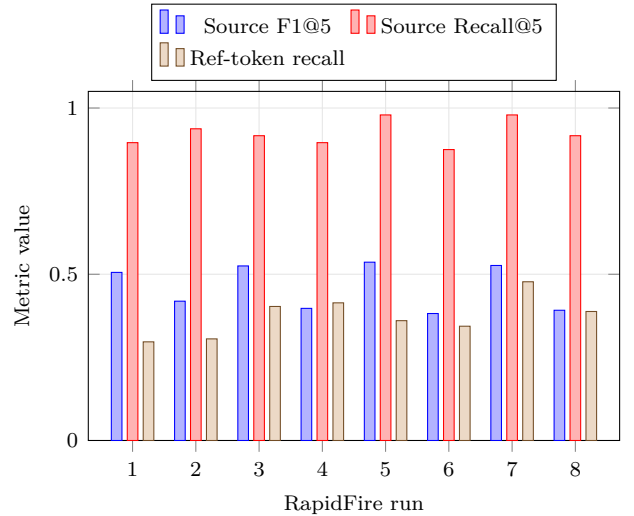


Figure 2: RapidFire dense RAG metric comparison over eight configs.

If I had one more week and more API budget, I would test a hybrid retriever that combines BM25 exact-match strength with BGE semantic recall, followed by a lightweight reranker only for ambiguous cases. I would also run the RapidFire configs on the full 45 released validation questions rather than the cost-controlled 24-sample subset and add an ablation that isolates prompt style from retriever changes.

5 AI Tools Statement

I used AI assistants, including Codex and ChatGPT-style tools, to inspect the codebase, draft implementation ideas, generate report structure, and debug scripts. I reviewed and edited the generated code and report text, ran validation checks, and made the final design decisions based on measured retrieval

and generation results. No released validation answers, hidden-test questions, or hidden-test-specific logic are hardcoded in the final pipeline.